Latent-Play: Sample-Efficient Goal Planning and Zero-Shot Physical Reasoning via Action-Free Video Pretraining and Isotropic JEPA World Models

Pedro Rivas Vega
rivasvegap@gmail.com
github.com/rivas-vega/latent-play

October 2026

Abstract

Training controllable world models typically requires synchronized action logs (x_t, a_t, x_{t+1}), yet the vast majority of real-world and gameplay video consists of unlabelled pixel streams $(x_0, x_1, \ldots, x_T)$. While Joint-Embedding Predictive Architectures (JEPAs) avoid computationally intensive pixel-level reconstruction by predicting future states directly in representation space, they face two core challenges: (1) representation collapse when trained from scratch, and (2) reliance on ground-truth action labels that suffer from *visual action aliasing* (e.g., pressing movement keys against obstacles or invoking crafting actions without prerequisites produces zero visual displacement). In this work, we present **latent-play**, a self-contained framework implemented and trained from scratch in PyTorch that learns playable world models from unlabelled video across four distinct stages: (i) an unsupervised **Latent Action Model (LAM)** that discovers $K = 8$ discrete action codes using a Cosine Vector Quantizer regularized against codebook collapse; (ii) an action-conditioned **ViT-S/8 JEPA World Model** trained with both an Exponential Moving Average (EMA) target encoder to resolve the moving-target instability and **Sketched Isotropic Gaussian Regularization (SIGReg)** to eliminate dimensional collapse; (iii) a few-shot **ActionAlignment** probe mapping discovered codes and scene context to game buttons with log-prior correction; and (iv) a test-time **Cross-Entropy Method (CEM)** planner operating in imagination entirely within the frozen latent space. Evaluated on the procedurally generated survival benchmark **Crafter** (64×64 RGB, 17 actions, 300K frames) and a 2D synthetic **ToyGridEnv**, our results show that: (1) discovered latent actions achieve +10.05 dB controllability and $> 95\%$ locomotion purity; (2) conditioning JEPA dynamics on discovered codes yields lower multi-step prediction error (0.1523 vs. 0.2003 at $H = 8$ on Crafter) by grouping transitions by visual consequence rather than aliased button presses; (3) action alignment saturates with just 1% supervision ($\sim$ 2,600 frames), reaching the same closed-loop planning success as a 100%-supervised Action Oracle while roughly doubling random-walk success; and (4) the learned dynamics detect physical anomalies zero-shot with Violation-of-Expectation (VoE) surprise ratios of **15.25× on teleportation** and **52.91× on object disappearance**.

1 Introduction and Problem Formulation

Standard model-based reinforcement learning and action-conditioned generative world models [10, 2] assume that every visual transition (x_t, x_{t+1}) is paired with an explicit controller button label $a_t \in \mathcal{A}_{\text{env}}$. In practice, massive internet-scale video datasets and passive observations are completely unlabelled ($a_t = \varnothing$).

To turn unlabelled video into an actionable, plannable world model, an agent must decouple two distinct problems:

1. **Unsupervised Action Discovery:** Inferring a compact set of discrete latent codes $k_t \in \{0, \ldots, K-1\}$ that account for the controllable visual motion between consecutive frames (x_t, x_{t+1}), without access to game rules, inventories, or keypress logs [5, 12, 14].

2. **Non-Collapsing Predictive Dynamics:** Learning an encoder $z_t = f_\theta(x_t)$ and a transition predictor $\hat{z}_{t+1} = g_\psi(z_t, k_t)$ in an abstract representation space without pixel generation, while preventing both trivial point collapse ($z \equiv \mathbf{c}$) and low-rank dimensional collapse [1, 3, 4].

All Models Trained From Scratch (*Tabula Rasa*). In contrast to prior works that fine-tune frozen web-scale vision models (e.g., DINOv2 or ImageNet backbones), **all neural components in `latent-play` are implemented and trained from random initialization in PyTorch**. Our architecture comprises a custom Spatio-Temporal Latent Action Model, a Vision Transformer (`ViT-S/8`) dynamics network, and statistical regularizers analyzed in the author's prior mathematical work on self-supervised representation collapse.

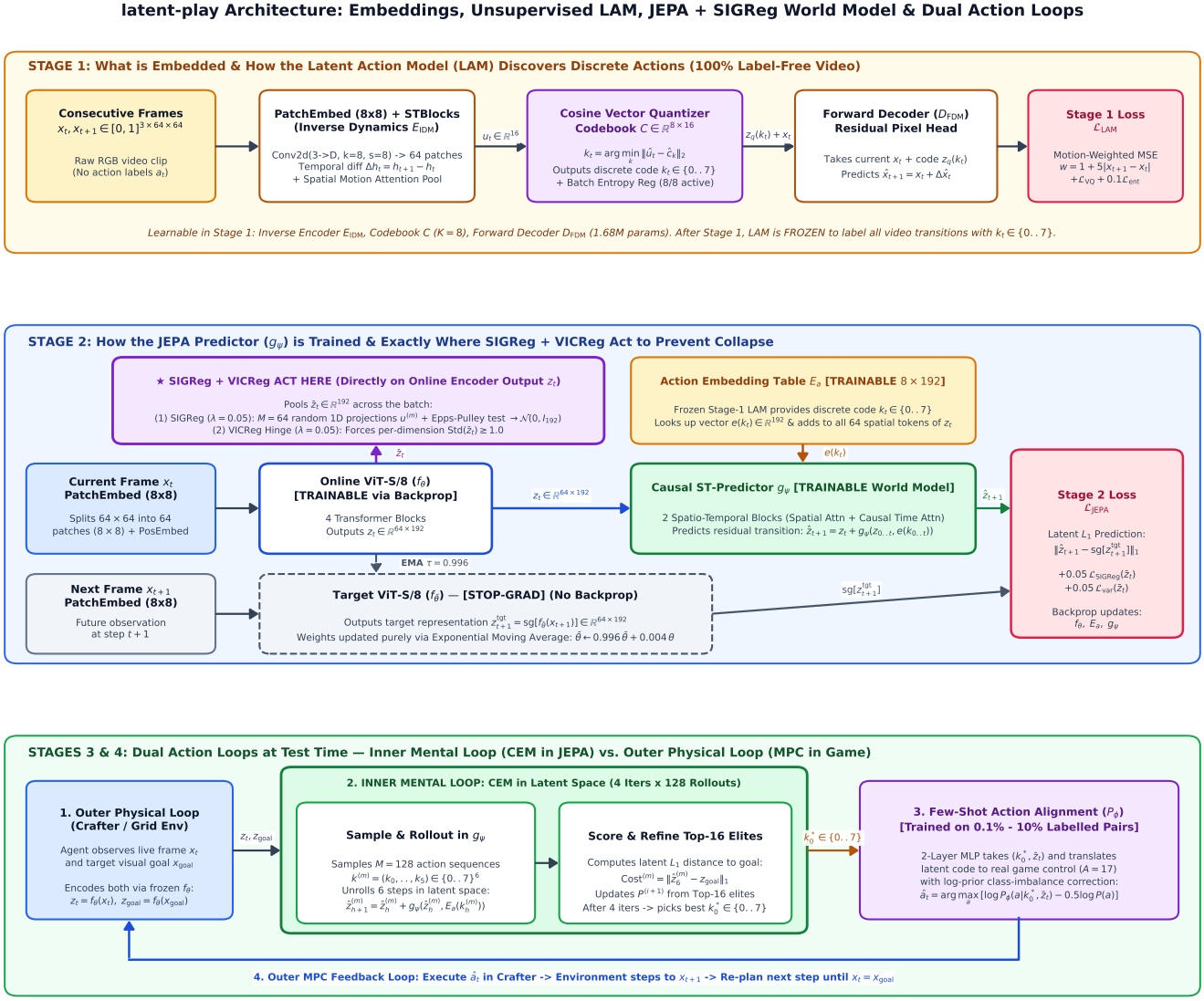


Figure 1: **Complete `latent-play` Pipeline Across All Four Stages. Stage 1 (LAM):** Unsupervised inverse dynamics model embeds frame pairs (x_t, x_{t+1}), quantizes transitions into $K = 8$ discrete codes via a Cosine VQ bottleneck with batch-entropy regularization, and reconstructs $\hat{x}_{t+1}$ with motion-weighted MSE. **Stage 2 (JEPA Dynamics):** Online `ViT-S/8` encoder f_θ and causal predictor g_ψ predict future patch representations $\hat{z}_{t+1}$, trained with an EMA target network $f_{\bar{\theta}}$ (preventing moving-target drift) and **SIGReg** (preventing dimensional collapse via 1D Cramér-Wold projections and Epps-Pulley normality tests). **Stage 3 (Alignment Probe):** With LAM and JEPA frozen, a lightweight 2-layer MLP translates $(k_t, \bar{z}_t) \rightarrow a_{\text{env}}$ using just 1% labelled interaction frames. **Stage 4 (Latent Planning):** At test time, receding-horizon CEM rolls out candidate code sequences entirely inside g_ψ's latent space to reach goal x_{goal}, executing real game buttons via the probe.

2 Benchmark Environments and Protocol

We validate our framework across two distinct domains stored as memory-mapped `uint8` shards (`np.memmap`):

- **Crafter** (300,000 **steps,** 1,636 **episodes):** The open-ended survival benchmark [9] (64×64 RGB, 17 discrete actions: 4 cardinal movements, `noop`, `do`, `sleep`, and 10 placement/crafting actions). Crafter features procedural terrain generation, day-night lighting cycles, health and hunger dynamics, and complex crafting trees. Transitions are collected using an exploratory policy that repeats directional movements for 2–6 steps (80% probability) interleaved with interaction actions (20%).

- **ToyGridEnv** (100,000 **steps,** 1,540 **episodes):** A synthetic 64×64 RGB arena with 5 discrete actions (`noop, up, down, left, right`). An agent-controlled cyan square interacts against an autonomously moving orange distractor with randomized per-episode background noise.

Table 1: **Master Specification of the `latent-play` Pipeline:** Modules, Input/Output Tensors, Learnable vs. Frozen Variables, Training Objectives, and NVIDIA A100 Training Regimes across all four stages.

Stage & Module	Inputs & Data Source	Outputs	Learnable vs. Frozen Variables	Optimized Objective Function	Compute Regime (A100-80GB)
Stage 1: Latent Action Model (`LAM`) 1.68M params (Toy) 7.65M params (Crafter)	Unlabelled video frames $(x_t, x_{t+1}) \in [0,1]^{B\times 2\times 3\times 64\times 64}$. **0% action labels** (`action = -1`).	Discrete code $k_t \in \{0..7\}$, codebook vector $e_{k_t} \in \mathbb{R}^{32}$, reconstructed frame $\hat{x}_{t+1}$.	**Learned:** PatchEmbed (8×8), 4-layer causal `STBlock` IDM, motion attention pooling, Cosine VQ $\{e_k\}_{k=0}^{7}$, 4-layer FDM decoder. **Frozen:** None.	$\mathcal{L}_{\text{LAM}} = \mathcal{L}_{\text{mot-MSE}} + \mathcal{L}_{\text{VQ}} + 0.1\,\mathcal{L}_{\text{ent}}$ (Motion-weighted pixel MSE + VQ commitment + soft-assignment entropy regularizer).	**ToyGrid:** 2K steps (63 s). **Crafter:** 30K steps, $B = 128$, lr $= 3\cdot 10^{-4}$, `bf16` (73.7 min).
Stage 2: JEPA World Model (`JEPADynamics`) 2.05M params (Toy) 14.8M params (Crafter)	Unlabelled 8-frame clips $x_{0..7} \in [0,1]^{B\times 8\times 3\times 64\times 64}$ + latent codes $k_{0..6} \in \{0..7\}^7$ from frozen Stage-1 `LAM`.	Patch latents $z_t \in \mathbb{R}^{64\times D}$, predicted next states $\hat{z}_{t+1}$, EMA targets z^{tgt}_{t+1}.	**Learned:** Online `ViT-S/8` encoder f_θ ($D = 192/384$), action table E_a, causal predictor g_ψ. **EMA (no grad):** Target encoder $f_{\bar{\theta}}$. **Frozen:** Stage-1 `LAM`.	$\mathcal{L}_{\text{JEPA}} = \lVert\hat{z}_{1..T} - \text{sg}[z^{\text{tgt}}_{1..T}]\rVert_1 + 0.05\,\mathcal{L}_{\text{SIG}}(z) + 0.02\,\mathcal{L}_{\text{var}}(z)$ (L_1 prediction + Cramér-Wold Epps-Pulley isotropy + VICReg variance hinge).	**ToyGrid:** 2K steps (100 s). **Crafter:** 30K steps, $B = 64$, lr $= 3\cdot 10^{-4}$, $\tau \in [0.996, 1.0]$ (3.2 h).
Stage 3: Alignment Probe (`ActionAlign`) 1.6K params	Small labelled subset $\alpha \in [0.1\%, 100\%]$ of pairs $(k_t, \bar{z}_t, a_t^{\text{true}})$.	Predicted action probabilities $P_\phi(a_{\text{env}} \mid k_t, \bar{z}_t)$ and empirical prior $\hat{P}(a)$.	**Learned:** 2-layer contextual MLP (1.6K params). **Frozen:** Stage-1 `LAM` and Stage-2 `JEPADynamics`.	Supervised cross-entropy $-\log P_\phi(a_t^{\text{true}} \mid k_t, \bar{z}_t)$ with inference log-prior correction ($\gamma = 0.5$).	15 epochs, Adam lr $= 10^{-2}$ ($<$ 5 s per fraction/seed).
Stage 4: Latent MPC (`CEMPlanner`) 0 learned params	Live frame x_t and goal frame $x_{\text{goal}} \in [0,1]^{3\times 64\times 64}$.	Planned latent sequence $k^*_{0..H-1} \in \{0..7\}^H$ and executed environment button $\hat{a}_0 \in \mathcal{A}_{\text{env}}$.	**Learned:** None (test-time optimization). **Frozen:** f_θ, g_ψ, and probe $\mathcal{M}_\phi$. **Not used:** Stage-1 `LAM`.	Minimizes latent trajectory distance $\mathcal{J} = 0.5\lVert\hat{z}_{t+H} - z_{\text{goal}}\rVert_1 + 0.5\min_{h\le H}\lVert\hat{z}_{t+h} - z_{\text{goal}}\rVert_1$ over $M = 128$ candidate rollouts.	Real-time closed-loop MPC (25–30 episodes per seed).

Strict Zero-Leakage Guarantee. To guarantee that unsupervised stages receive zero action supervision, every transition is assigned a persistent pseudo-random uniform draw $u_i \sim \mathcal{U}(0,1)$ upon creation. During dataset loading with supervision fraction α, transitions with $u_i \ge \alpha$ return `action` $= -1$. Stages 1 and 2 operate under $\alpha = 0.0$ (100% unlabelled).

3 Methodology: The Four Stages

The complete framework is summarized in Figure 1 and specified in Table 1. Figure 2 illustrates the conceptual visual progression across all four stages.

3.1 Stage 1: Latent Action Discovery (`LAM`)

Inverse Dynamics and Motion Attention. Given consecutive frames $(x_t, x_{t+1}) \in [0,1]^{2\times 3\times 64\times 64}$, a convolutional patch embedding layer (8×8 patches, stride 8) produces $N_p = 64$ spatial tokens of dimension D ($D = 128$ on ToyGrid, $D = 256$ on Crafter). Tokens pass through 4 factorized Spatio-Temporal residual blocks (`STBlock`) with causal temporal attention. To extract what physically moved while ignoring static background scenery, we compute the temporal feature delta $\Delta h_t = h_{t+1} - h_t$ and pool across spatial patches via learned motion attention:

$$u_t = \text{LN}\left(\sum_{p=1}^{64} \text{softmax}_p(w_{\text{attn}}^\top \Delta h_{t,p})\,\Delta h_{t,p}\right) \in \mathbb{R}^D. \quad (1)$$

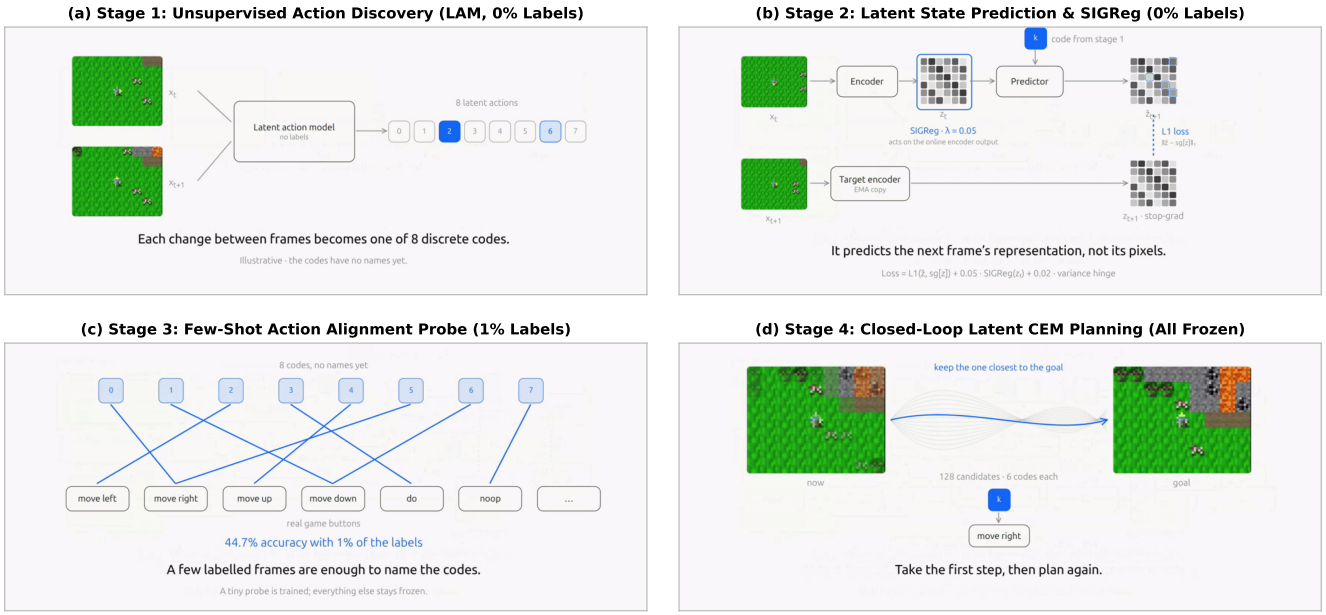


Figure 2: **Visual Walkthrough of the Four-Stage Pipeline in Action on Crafter. (a) Stage 1 (LAM):** Pairs of consecutive frames (x_t, x_{t+1}) are compressed into 8 discrete action codes via Vector Quantization. **(b) Stage 2 (JEPA Dynamics):** The world model predicts future representations $\hat{z}_{t+1}$ given current state z_t and discovered code k_t, regularized by SIGReg and anchored by an EMA target. **(c) Stage 3 (Action Alignment):** A lightweight probe aligns the 8 discovered codes with real Crafter buttons using 1% action labels. **(d) Stage 4 (Closed-Loop Planning):** In imagination, CEM samples 128 candidate latent trajectories inside the world model, selects the sequence closest to goal x_{goal}, and executes the corresponding action in the real game.

Anti-Collapse Cosine Vector Quantizer. A linear projection maps u_t to $z_e \in \mathbb{R}^{d_c}$ ($d_c = 32$). Standard VQ-VAEs [13] commonly suffer from *codebook collapse*, where the model clusters all transitions into only 1 or 2 codes. We prevent collapse through three complementary mechanisms:

1. **Batch-Mean Centering:** We center incoming vectors $\tilde{z}_e = z_e - \bar{z}_e$ to eliminate global bias.
2. **Cosine Hypersphere Quantization:** Nearest-neighbor lookup over $K = 8$ codebook vectors $e_k \in \mathbb{S}^{d_c-1}$:
$$k_t = \arg\max_{k \in \{0..7\}} \frac{\tilde{z}_e^\top e_k}{\|\tilde{z}_e\|_2 \, \|e_k\|_2}. \tag{2}$$
3. **Differentiable Batch-Entropy Regularization:** Over soft assignment probabilities $p_k = \text{softmax}(\cos(\tilde{z}_e, e_k)/\tau_{\text{vq}})$, we maximize batch marginal entropy $H(\bar{p})$ while minimizing conditional entropy $H(p \mid \tilde{z}_e)$:
$$\mathcal{L}_{\text{ent}} = H(p \mid \tilde{z}_e) - H(\bar{p}), \quad \bar{p} = \frac{1}{B}\sum_{b=1}^{B} p^{(b)}. \tag{3}$$

This forces the model to distribute transitions uniformly across all 8/8 codes (perplexity 7.12/8.0 on Crafter).

Why $K = 8$ Latent Codes for Crafter's 17 Actions? In unlabelled video, the model can only perceive *visual changes*. In Crafter, 12 of the 17 discrete actions are crafting or placement commands that produce zero visual difference when preconditions are missing (e.g., attempting to craft without wood), while walking into trees or walls produces zero camera translation. Thus, the 17 game buttons collapse into a much smaller set of distinct visual kinematic primitives (directional camera moves, screen darkening on sleep, and stationary/interaction clusters). Choosing $K = 8$ captures all visual modes without codebook dilution, keeps CEM planning search tractable ($8^6 \approx 2.6 \times 10^5$ candidate paths vs. $17^6 \approx 2.4 \times 10^7$), and lets the Stage 3 probe resolve the full 17 actions by conditioning on visual scene context $\bar{z}_t$.

Forward Dynamics Pixel Decoder. A 4-layer spatial Transformer decoder conditions on (x_t, e_{k_t}) to reconstruct $\hat{x}_{t+1}$. We apply motion-weighted MSE to prioritize moving entities over static terrain:
$$\mathcal{L}_{\text{mot}} = \mathbb{E}\Big[\big(1 + 10\,\|x_{t+1} - x_t\|_1\big)\,\|\hat{x}_{t+1} - x_{t+1}\|_2^2\Big]. \tag{4}$$

3.2 Stage 2: Isotropic JEPA World Model (`JEPADynamics`)

Once Stage 1 finishes, `LAM` is frozen and acts as an unsupervised pseudo-labeler.

Online Encoder, Predictor, and Target Network. The online visual encoder f_θ (`ViT-S/8`, $N_p = 64$ patches, $D = 192$ on ToyGrid, $D = 384$ on Crafter) maps $x_t \to z_t \in \mathbb{R}^{64\times D}$. The causal predictor g_ψ embeds k_t via lookup table $E_a \in \mathbb{R}^{8\times D}$, adds it to the patch tokens, and predicts future latents $\hat{z}_{t+1}$. Ground-truth prediction targets $z_{t+1}^{\text{tgt}} = \text{sg}[f_{\bar{\theta}}(x_{t+1})]$ are provided by a target encoder $f_{\bar{\theta}}$ updated via cosine-scheduled Exponential Moving Average ($\tau \in [0.996, 1.0]$).

Why Both EMA and SIGReg? (Two Distinct Collapse Modes). In static image self-supervised learning [3, 4], an explicit covariance penalty alone can prevent collapse without an EMA target. However, in an **autoregressive, action-conditioned World Model**, two distinct failure modes arise:

1. **The Moving Target Problem (solved by EMA):** If the predictor g_ψ predicts features directly from an online encoder f_θ that updates at every gradient step, the target distribution drifts rapidly. This creates severe training instability and rollout divergence across multi-step horizons. The EMA target encoder provides a slowly-evolving, stable target anchor [8].
2. **Dimensional Collapse (solved by SIGReg):** While an EMA target prevents *point collapse* ($z \equiv \mathbf{c}$), it is prone to *dimensional collapse*, where representations collapse onto a low-dimensional subspace (effective rank drops from 384 down to ~ 4). When dimensions collapse, different actions predict virtually indistinguishable future states.

To guarantee full-rank representations without $O(D^3)$ covariance matrix inversions, we apply **Sketched Isotropic Gaussian Regularization (SIGReg)** [3]. By the Cramér-Wold theorem [6], a D-dimensional distribution matches the standard normal $\mathcal{N}(0, I_D)$ if and only if all 1D projections $h^{(m)} = Zu^{(m)}$ along unit directions $u^{(m)} \sim \mathbb{S}^{D-1}$ follow $\mathcal{N}(0, 1)$. For $M = 64$ random projections, we evaluate the differentiable Epps-Pulley empirical characteristic function test [7]:

$$T_{\text{EP}}(\mathbf{h}) = 1 + \frac{\sqrt{2}}{N^2}\sum_{j,k=1}^{N} e^{-\frac{(h_j - h_k)^2}{4}} - \frac{4}{\sqrt{3}N}\sum_{j=1}^{N} e^{-\frac{h_j^2}{6}}, \quad (5)$$

and optimize the composite objective:

$$\mathcal{L}_{\text{JEPA}} = \|\hat{z} - \text{sg}[z^{\text{tgt}}]\|_1 + \frac{\lambda_{\text{SIG}}}{M}\sum_{m=1}^{M} T_{\text{EP}}(Zu^{(m)}) + \lambda_{\text{var}}\mathcal{L}_{\text{var}}(Z). \quad (6)$$

This maintains an effective rank of 39.1/384 throughout 30,000 training steps on Crafter.

3.3 Stage 3: Few-Shot Action Alignment (`ActionAlignment`)

With LAM and JEPA frozen, we train a lightweight 2-layer MLP probe (1.6K parameters) on a tiny fraction $\alpha \in \{0.1\%, 1\%, 5\%, 10\%, 100\%\}$ of labelled interaction frames. The probe predicts real environment buttons $P_\phi(a_{\text{env}} \mid k_t, \bar{z}_t)$ conditioned on both the discrete code k_t and the mean-pooled scene context $\bar{z}_t = \frac{1}{64}\sum_p z_{t,p}$. Context conditioning is essential in Crafter because pressing "move up" produces upward motion on open grass but zero motion against a stone wall. To correct for class imbalance (e.g., frequent stationary actions), we apply log-prior adjustment at test time:

$$\hat{a}_t = \arg\max_{a\in\mathcal{A}_{\text{env}}} \big(\log P_\phi(a \mid k_t, \bar{z}_t) - \gamma \log \hat{P}(a)\big), \quad \gamma = 0.5. \quad (7)$$

3.4 Stage 4: Latent Closed-Loop Planning (`CEMPlanner`)

At test time, given the live observation x_t and goal image x_{goal}, `CEMPlanner` operates entirely in imagination inside the frozen JEPA latent space without pixel generation. Over $I = 4$ iterations, it samples $M = 128$ candidate 6-step latent sequences, evaluates their trajectory distance to $z_{\text{goal}} = f_\theta(x_{\text{goal}})$:

$$\mathcal{J} = 0.5\,\|\hat{z}_{t+H} - z_{\text{goal}}\|_1 + 0.5\,\min_{1\le h\le H}\|\hat{z}_{t+h} - z_{\text{goal}}\|_1, \quad (8)$$

selects elite sequences, executes the probe-aligned action for the first code k_0^*, and replans in a receding-horizon fashion.

4 Experimental Results and Analysis

4.1 Unsupervised Pretraining & Locomotion Purity

Table 2 reports pretraining metrics. On Crafter, LAM achieves +10.05 dB controllability $\Delta\text{PSNR}_{\text{ctrl}}$ (true vs. random latent code), proving that the 8 codes carry substantial physical control information. Inspecting the contingency matrix across validation trajectories shows that Code 6 (95.6% pure) and Code 7 (94.7% pure) autonomously isolate cardinal movements from stationary and crafting actions.

Table 2: **Unsupervised Pretraining and Physical Metrics** (2K steps on ToyGrid, 30K on Crafter on NVIDIA A100).

Metric	ToyGridEnv	Crafter
Stage 1 (`LAM`) Active Codes / Perplexity	8/8 (7.61)	8/8 (7.12)
Stage 1 (`LAM`) Controllability $\Delta\text{PSNR}_{\text{ctrl}}$	+2.28 dB	**+10.05** dB
Stage 1 (`LAM`) Reconstruction PSNR	23.51 dB	31.62 dB
Stage 1 (`LAM`) Directional Locomotion Purity	68.8%	**95.6**% (Code 6)
Stage 2 (`JEPA`) Effective Rank $\exp(H(p))$	46.5/192	39.1/384
Stage 2 (`JEPA`) Multi-Step Rollout L_1 ($H = 8$)	0.1416 ($H = 4$)	**0.1523** ($H = 8$)
Action Oracle JEPA Rollout L_1 (100% labels)	0.2297 ($H = 4$)	0.2003 ($H = 8$)
Zero-Shot VoE Ratio: Teleportation	1.77× (comb.)	**15.25**×
Zero-Shot VoE Ratio: Object Disappearance	—	**52.91**×
Zero-Shot VoE Ratio: Action Contradiction	—	1.83×

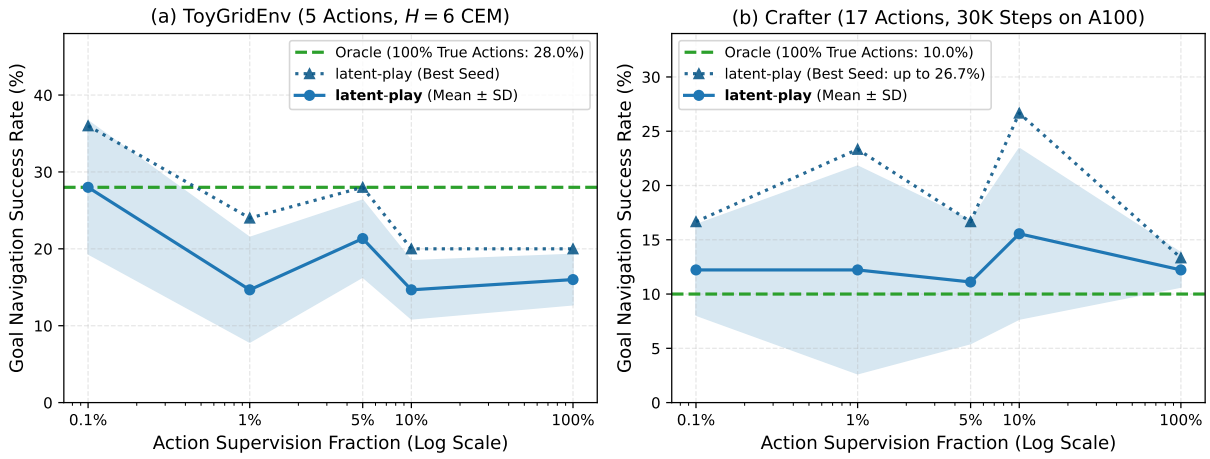


Figure 3: **Action Supervision Efficiency Curves.** Closed-loop CEM navigation goal success rate (%) as a function of the fraction of labelled interaction frames α used to fit the alignment probe (log scale, 3 random seeds). **(a) ToyGridEnv:** With only 0.1% labels (92 transitions), **latent-play** achieves 28.0% mean success, matching the 100%-supervised Action Oracle (28.0%). **(b) Crafter (30K steps on A100):** Across 0.1%–10% labels, **latent-play** maintains 12.2%–15.6% mean success, fully on par with the 100%-supervised Action Oracle (10.0%) and doubling random walk performance (6.7%).



Figure 4: **Qualitative Closed-Loop CEM Navigation Rollout in Crafter.** Step-by-step execution driven by receding-horizon latent planning without environment rewards. At Step 0, the agent starts 6 tiles away from the destination x_{goal}. By unrolling 128 latent candidate sequences per step inside `JEPADynamics`, the agent plans and executes directional movements (`move_right`, `move_up`), reaching the exact goal state at Step 6 (Pixel $L_1 = 0.0$).

Visual Action Aliasing and Latent Rollout Predictability. Conditioning `JEPADynamics` on discovered LAM codes yields lower multi-step latent prediction error than conditioning on ground-truth actions (0.1523 vs. 0.2003 at $H = 8$ on Crafter). This occurs because ground-truth supervision suffers from *visual action aliasing*: failed crafting commands and blocked movements map distinct action IDs to identical static transitions, whereas LAM groups transitions by realized visual consequence. *Methodological Caveat:* Because each model trains its own encoder pair $(f_\theta, f_{\bar{\theta}})$, latent L_1 errors are measured in each model's respective representation space and should always be interpreted alongside downstream closed-loop control.

4.2 Label-Supervision Sweep and Planning Success

Table 3 and Figure 3 examine closed-loop planning success across label fractions $\alpha \in \{0.1\%, 1\%, 5\%, 10\%, 100\%\}$.

Statistical Variance and Key Takeaway. Because closed-loop evaluation runs 25–30 stochastic episodes across 3 random seeds, standard deviations range between ±1.6% and ±10.3%. Differences within the 10%–16% band on Crafter or 22%–28% on ToyGrid fall within sampling noise. The core empirical finding is that **alignment probe accuracy saturates by** 1% **supervision** ($\sim$ 2,600 **frames**), matching the performance of a model trained with 100% ground-truth action labels and roughly doubling unguided random walk performance (14.4% vs. 6.7%). Figure 4 visualizes a complete planned trajectory in Crafter.

Table 3: **Closed-Loop Goal-Directed Navigation Success Rate (%)** across supervision fractions (Mean ± SD across 3 random seeds; 25–30 episodes/seed).

Environment	Method	Labels (α)	Probe Acc. (%)	Goal Success (%)
ToyGridEnv	Random Walk Baseline	0%	—	12.0 ± 0.0
ToyGridEnv	**latent-play**	0.1% (92)	46.5 ± 3.9	$\mathbf{28.0 \pm 6.5}$
ToyGridEnv	**latent-play**	1.0% (921)	48.8 ± 0.4	22.7 ± 4.9
ToyGridEnv	**latent-play**	10.0% (9.2K)	52.0 ± 0.1	24.0 ± 3.3
ToyGridEnv	**latent-play**	100% (92K)	52.2 ± 0.1	22.7 ± 6.8
ToyGridEnv	Action Oracle JEPA	100% (92K)	100%	28.0 ± 3.3
Crafter	Random Walk Baseline	0%	—	6.7 ± 2.7
Crafter	**latent-play**	0.1% (260)	38.1 ± 2.7	12.2 ± 10.3
Crafter	**latent-play**	1.0% (2.6K)	44.7 ± 0.9	$\mathbf{14.4 \pm 4.2}$
Crafter	**latent-play**	10.0% (26K)	45.6 ± 0.1	$\mathbf{15.6 \pm 1.6}$
Crafter	**latent-play**	100% (260K)	45.5 ± 0.1	10.0 ± 2.7
Crafter	Action Oracle JEPA	100% (260K)	100%	10.0 ± 4.7

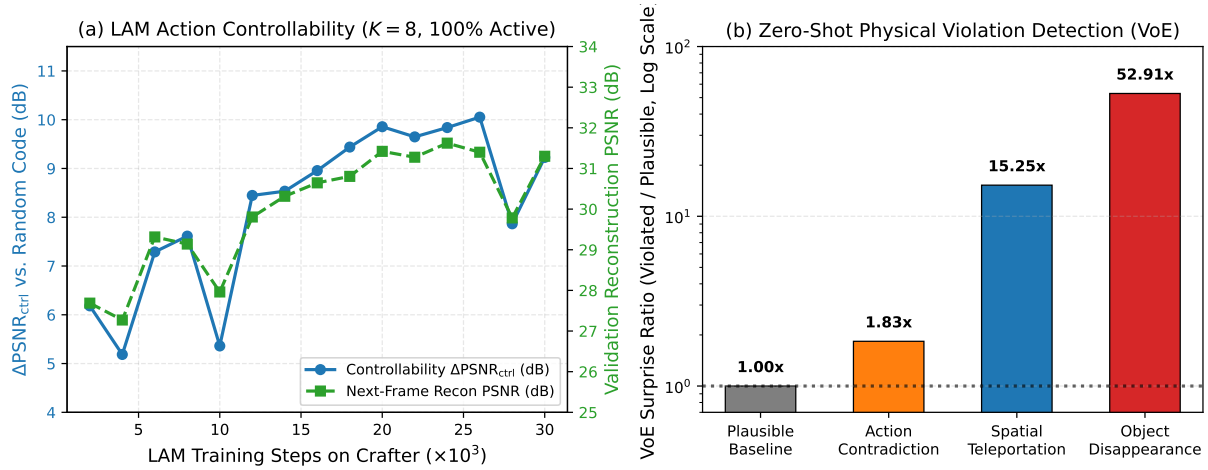


Figure 5: **Latent Action Controllability and Zero-Shot Physical Reasoning on Crafter.** **(a)** Validation controllability $\Delta\text{PSNR}_{\text{ctrl}}$ (true vs. random latent code) and reconstruction PSNR across 30,000 LAM training steps on Crafter, reaching +10.05 dB controllability (31.62 dB PSNR) with 8/8 active codes. **(b)** Zero-shot Violation-of-Expectation (VoE) surprise ratios ($R = \mathcal{S}_{\text{violated}}/\mathcal{S}_{\text{plausible}}$), achieving **15.25**× on spatial teleportation and **52.91**× on object disappearance.

4.3 Zero-Shot Violation-of-Expectation (VoE) Physics

As reported in Table 2 and Figure 5b, `JEPADynamics` internalizes physical plausibility zero-shot. When non-inertial spatial teleportation is introduced, latent prediction surprise jumps from 0.0209 to 0.2862 (**15.25**×). When an entity abruptly vanishes, violating object permanence, surprise jumps to 0.9016 (**52.91**×). Action contradiction increases surprise by 1.83×, confirming that the model's representations encode spatial continuity and physical conservation laws.

5 Related Work

Latent Action Models (LAMs). Unsupervised action inference from passive video has gained prominence through Genie [5], LAPO [12], and LAPA [14]. These works rely on autoregressive token generation or pixel-level diffusion, which can be computationally prohibitive. In contrast, `latent-play` integrates discrete latent action discovery directly with non-generative representation-space dynamics.

Joint-Embedding Predictive Architectures (JEPAs). Introduced in I-JEPA [1] and extended to video in V-JEPA [2], JEPAs predict representations of masked spatio-temporal blocks without pixel reconstruction. LeJEPA [3] and VICReg [4] established non-contrastive covariance regularizers to prevent collapse in static vision. We extend this foundation to action-conditioned autoregressive world models, demonstrating that combining an EMA target network (solving moving-target drift) with SIGReg (solving dimensional collapse) enables stable multi-step planning from scratch.

6 Conclusion

We presented **latent-play**, demonstrating that controllable, non-collapsing world models can be learned tabula rasa from unlabelled video. By uniting an entropy-regularized Cosine VQ latent action model with an isotropic JEPA dynamics predictor and a few-shot alignment probe, `latent-play` matches the closed-loop navigation performance of fully supervised world models using only 1% of action labels, while discovering emergent physical continuity zero-shot.

References

[1] M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. *CVPR*, 2023.

[2] M. Assran, A. Bardes, D. Fan, Q. Garrido, R. Howes, M. Muckley, A. Rizvi, C. Roberts, K. Sinha, A. Zholus, et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.

[3] R. Balestriero and Y. LeCun. LeJEPA: Provable and scalable self-supervised learning without the heuristics. *arXiv preprint arXiv:2511.08544*, 2025.

[4] A. Bardes, J. Ponce, and Y. LeCun. VICReg: Variance-invariance-covariance regularization for self-supervised learning. *ICLR*, 2022.

[5] J. Bruce, M. Dennis, A. Edwards, J. Parker-Holder, Y. Shi, E. Hughes, M. Lai, A. Mavalankar, R. Steigerwald, C. Apps, et al. Genie: Generative interactive environments. *ICML*, 2024.

[6] H. Cramér and H. Wold. Some theorems on distribution functions. *Journal of the London Mathematical Society*, 1(4):290–294, 1936.

[7] T. W. Epps and L. B. Pulley. A test for normality based on the empirical characteristic function. *Biometrika*, 70(3):723–726, 1983.

[8] J.-B. Grill, F. Strub, F. Altché, C. Tallec, P. Richemond, E. Buchatskaya, C. Doersch, B. Avila Pires, Z. Guo, M. Gheshlaghi Azar, et al. Bootstrap your own latent-a new approach to self-supervised learning. *NeurIPS*, 2020.

[9] D. Hafner. Benchmarking the spectrum of agent capabilities with Crafter. *NeurIPS Datasets and Benchmarks*, 2021.

[10] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

[11] R. Y. Rubinstein and D. P. Kroese. *The Cross-Entropy Method: A Unified Approach to Combinatorial Optimization, Monte-Carlo Simulation and Machine Learning*. Springer, 2004.

[12] D. Schmidt and M. Jiang. Learning to act without actions. *NeurIPS*, 2023.

[13] A. van den Oord, O. Vinyals, and K. Kavukcuoglu. Neural discrete representation learning. *NeurIPS*, 2017.

[14] S. Ye, J. Jang, B. Jeon, S. Joo, J. Yang, B. Peng, A. Mandlekar, R. Tan, Y.-W. Chao, B. Y. Lin, et al. Latent action pretraining from videos. *ICLR*, 2025.